



# MediConnect Telehealth & AI Platform

Comprehensive Full-Stack Architecture, Real-Time WebSockets, WebRTC P2P Video, & Technical Interview Master Guide

MERN STACK

WEBRTC 1080P P2P

SOCKET.IO EVENT BUS

GOOGLE GEMINI 2.0 AI

RESEND/SENDPULSE SMTP

RBAC SECURITY

## 1. Executive Summary & Problem Solved

**MediConnect** is an enterprise-grade virtual care ecosystem engineered to solve real-world healthcare bottlenecks: diagnostic delays, geographical care disparities, and fragmented prescription workflows.

### Core Capabilities:

- **AI Clinical Symptom Triage:** Google Gemini 2.0 Flash categorizes user symptoms into urgency tiers (*Emergency, High, Moderate, Low*) and pairs them with verified specialists.
- **Encrypted WebRTC Telehealth Suite:** Sub-100ms peer-to-peer browser video consultations with in-call clinical notes & chat.
- **Real-Time Slot Scheduling Engine:** Collision prevention, atomic bookings, cancelled slot recycling, and live socket alerts.
- **Digital Prescription Studio:** Doctor-signed multi-medication digital prescriptions with dosage, frequency, and instant email delivery.

## 2. Technology Stack & Architectural Justifications

| Technology                    | Role in MediConnect                    | Why Chosen Over Alternatives                                                                                                                                                            |
|-------------------------------|----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| React 18 + Vite               | Single Page Application (SPA) Frontend | <b>vs. Next.js / CRA:</b> Instant HMR and rollup compilation. SPA runtime preserves persistent WebRTC streams and Socket connections without SSR hydration mismatch.                    |
| Node.js & Express.js          | Event-Driven REST API Gateway          | <b>vs. Django / Spring Boot:</b> Non-blocking async I/O efficiently manages thousands of concurrent WebSocket signaling handshakes with minimal memory.                                 |
| MongoDB & Mongoose            | Document Database & ODM                | <b>vs. PostgreSQL / SQL:</b> Healthcare data has nested polymorphic structures (prescriptions, doctor qualifications). Rich indexing on { doctor, date, timeSlot } prevents collisions. |
| Socket.io                     | Bidirectional Real-Time Signaling      | <b>vs. Raw WebSockets / Polling:</b> Automatic reconnects, heartbeat ping-pong, and partitioned rooms (user- <code>{id}</code> personal queues and roomId video channels).              |
| Browser-Native WebRTC         | P2P HD Video & Audio Stream            | <b>vs. Twilio / Agora SDKs:</b> Zero per-minute third-party licensing fees, sub-100ms latency, and direct 256-bit DTLS/SRTP browser encryption.                                         |
| Google Gemini 2.0 Flash       | Clinical AI Symptom Triage             | <b>vs. OpenAI GPT-4:</b> High-speed sub-second inference, strict JSON schema compliance, paired with a deterministic heuristic fallback engine.                                         |
| Nodemailer + Resend/SendPulse | Transactional Email Engine             | <b>vs. Third-Party Webhooks:</b> Standardized SMTP relay interface supporting Resend, SendPulse, or Gmail App Passwords with zero code modifications.                                   |

### 3. Database Schemas & Data Relationships

---

The platform is modeled across 5 specialized MongoDB collections:

#### 1. User Schema (User.js)

Stores credentials, personal profiles, medical vitals, and password reset tokens.

```
const UserSchema = new mongoose.Schema({ name: { type: String, required: true }, email: { type: String, required: true, unique: true }, password: { type: String, required: true }, // bcrypt hashed (salt=10) role: { type: String, enum: ['PATIENT', 'DOCTOR', 'ADMIN'], default: 'PATIENT' }, phone: String, avatar: String, bloodGroup: String, allergies: String, emergencyContact: String, resetPasswordToken: String, resetPasswordExpires: Date }, { timestamps: true });
```

#### 2. Doctor Profile Schema (DoctorProfile.js)

1-to-1 extension of the Doctor User containing verification status and hourly rates.

```
const DoctorProfileSchema = new mongoose.Schema({ user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true }, specialty: { type: String, index: true }, qualification: String, experienceYears: Number, hourlyFee: Number, hospital: String, bio: String, licenseNumber: { type: String, required: true, unique: true }, status: { type: String, enum: ['PENDING', 'APPROVED', 'REJECTED'], default: 'PENDING' }, rating: { type: Number, default: 4.9 }, reviewCount: { type: Number, default: 12 } });
```

#### 3. Appointment & Prescription Schemas

Manages scheduled consultations, meeting rooms, and issued medication arrays.

```
// Appointment (with composite collision prevention index) AppointmentSchema.index({ doctor: 1, date: 1, timeSlot: 1, status: 1 }); // Prescription schema with embedded medicines array medicines: [{ name: String, dosage: String, frequency: String, duration: String }]
```

### 4. WebRTC Peer-to-Peer Video & Real-Time Signaling

---

WebRTC creates a direct, peer-to-peer audio/video tunnel between Patient and Doctor using the browser's native media engine:

### Step-by-Step Signaling Handshake:

1. **Room Join:** Patient and Doctor connect via Socket.io to `join-room { roomId }`.
2. **Offer Creation:** The first peer calls `pc.createOffer()` and emits `webrtc-offer` over Socket.io.
3. **Answer Generation:** The receiving peer calls `pc.setRemoteDescription(offer)`, calls `pc.createAnswer()`, and replies with `webrtc-answer`.
4. **NAT Traversal (STUN):** Both browsers query Google STUN servers (`stun:stun.l.google.com:19302`) to discover public IP/port candidates and exchange them via `webrtc-candidate`.
5. **Direct Media Stream:** Once candidates match, high-definition 1080p video streams directly peer-to-peer with 256-bit DTLS/SRTP encryption.

## 5. AI Clinical Triage & Safety Fallback Engine

The AI engine translates raw, unstructured patient text into structured medical recommendations:

1. Gemini 2.0 JSON-Schema Prompting: Enforces exact JSON keys: summary, suggestedSpecialty, urgencyLevel, recommendedAction, and keyQuestions.

2. Deterministic Heuristic Fallback Engine: If the Gemini API experiences network limits, a built-in medical rule engine intercepts symptoms safely:

- Chest tightness / breath shortness → **Cardiology** (High / Emergency → Hospital ER Alert)
- Severe migraine / photophobia → **Neurology** (Moderate / High)
- Skin rash / eczema → **Dermatology** (Low)
- Joint pain / bone fracture → **Orthopedics** (Moderate)

## 6. Key Engineering Challenges & Solutions

| Challenge                     | Root Cause                                                                 | Implemented Solution                                                                                                                                              |
|-------------------------------|----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| WebRTC Race Conditions        | Remote peer received ICE candidates before initializing RTCPeerConnection. | Implemented stateful sequencing in VideoConsultationPage.jsx: connection is created upon user-joined and candidates are buffered until remote description is set. |
| Concurrent Booking Collisions | Two patients booking the same slot at the exact same millisecond.          | Enforced composite database index { doctor, date, timeSlot } + atomic recycling of cancelled slot records.                                                        |
| Password Token Security Leaks | Initial API returning reset tokens in HTTP response bodies.                | Hashed tokens with SHA-256 in MongoDB (+1 hour expiry) and delivered tokens strictly via verified SMTP email.                                                     |
| Zero-Downtime Resilience      | Cloud DB cold-starts causing evaluation failures.                          | Integrated an in-memory data store fallback in server.js routing traffic seamlessly when database is unreachable.                                                 |

## 7. Senior-Level Interview Preparation Master Guide

---

### Q1: How does WebRTC connect two users across firewalls and NATs?

WebRTC uses Session Description Protocol (SDP) and ICE. Our Socket.io server acts as the signaling channel. Peers exchange SDP Offers and Answers. Simultaneously, STUN servers resolve the public IP/port mappings (NAT traversal). Once candidates match, an encrypted P2P media stream is established directly between browsers.

### Q2: Why use WebSockets alongside REST APIs instead of WebSockets for everything?

REST APIs are ideal for idempotent CRUD operations (authentication, profile updates, medical records) where HTTP caching, status codes (200, 401, 500), and rate-limiting middleware provide security. Socket.io is used specifically for high-frequency real-time events: WebRTC signaling, in-call chat, and instant dashboard push notifications.

### Q3: How is Role-Based Access Control (RBAC) enforced in the backend?

We enforce defense-in-depth security: protect middleware validates the JWT token signature and decodes user context. `authorize('DOCTOR', 'ADMIN')` checks permissions. In controllers, we enforce object-level ownership (e.g., doctors can only prescribe for appointments they attend; patients cannot modify appointment statuses).

### Q4: How would you scale this platform to 100,000 concurrent video calls?

- 1) Transition from mesh P2P to a Selective Forwarding Unit (SFU) like Mediasoup or LiveKit to optimize client bandwidth.
- 2) Implement a Redis Adapter for Socket.io to scale WebSocket servers horizontally behind an NGINX load balancer.
- 3) Shard MongoDB by `doctorId` and `date` with Redis caching for doctor directories.